

# Payout Gateway Integration, Payments Dashboard & Payment Audit — Design Spec

---

**Date:** 2026-06-10 **Owner:** Nick (Sagara Corp) **Status:** Approved design → pending implementation plan **Maps to:** Sprints 16, 17, 18, 19

---

## 1. Problem & Goal

---

Today the platform can *charge* members (Spreedly vault + routing engine, fully built) but it cannot really *pay ambassadors out*. The current payout path ( `PayoutHandler` , `POST /api/v1/billing/wallets/payout` ) only marks `CommissionEarning` rows as `Paid` and credits an internal eWallet balance — **no money ever leaves to an external provider** (i-payout / Volet), and there is no real interaction with those gateways.

This work delivers real outbound payouts through external payout gateways, an admin dashboard to process pending payments, and a tamper-evident payment audit trail designed to survive legal disputes.

### The dispute the audit must defend against

An ambassador is paid to a gateway/account, then later changes their payout method (e.g. swaps their crypto address) and claims they were never paid. The audit must prove **which gateway** and **which exact account** received **which grouped earnings**, frozen at the moment of payment, plus the raw gateway responses and a tamper-evident, externally-anchored receipt.

---

## 2. Scope

---

In scope (4 sequenced iterations):

- Iteration 1 / Sprint 16 — Payout Gateway + audit core (foundation).** Outbound gateway abstraction (subscribe / validate / get-balance / disburse), eWallet (i-payout) + Volet implementations behind a switchable stub, lazy auto-subscribe payout flow gated on gateway confirmation, the immutable `PayoutAttempt` audit record, per-gateway minimum thresholds. **Also lays the bulk-payout data model** ( `PayoutBatch` , `DisbursementMode` , earnings reservation) so Iteration 4 needs no schema rework.
- Iteration 2 / Sprint 17 — Payments dashboard.** Admin dashboard: pending-payout candidates by threshold + process date, per-gateway pending/paid-today stats, gateway filter, individual (online) pay action, per-row result icon + rejection-reason popup.
- Iteration 3 / Sprint 18 — Receipts, authenticity & audit screen.** Receipt PDF + email (auto-send configurable + resend), internal Merkle hash-chain + external blockchain anchoring of the receipt, audit screen with date/ambassador/gateway filters and full gateway-response history.
- Iteration 4 / Sprint 19 — Bulk CSV payout path.** For high-volume promo windows where per-member API calls don't scale: export a per-gateway CSV of pending payouts to upload to the gateway portal, then reconcile back. Two reconciliation modes — per-row via the gateway's results file, with a manual bulk mark-paid fallback. Reuses the same success path (mark Paid + receipt + hash-chain + email) as online payouts.

Out of scope: weighted routing / fallback chains for payouts (each ambassador pays via their preferred gateway — no routing engine needed); production gateway credentials (everything runs in switchable simulated mode until creds exist); frontend visual polish beyond the functional Blazor AdminWeb pages (delegated to Jazmin-FrontEnd).

---

### 3. Decisions locked during brainstorming

| Topic                     | Decision                                                                                                                                                                                                                                                                             |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Meaning of "subscription" | Register/onboard the ambassador's payout account at the gateway (account creation API). Not recurring billing.                                                                                                                                                                       |
| Disbursement              | Real money transfer. On gateway success, mark the member's pending earnings with <code>PaymentDate ≤ processDate</code> as <code>Paid</code> . On failure they stay <code>Pending</code> and the error is audited.                                                                   |
| <code>get-balance</code>  | The <b>ambassador's</b> balance in their own gateway account.                                                                                                                                                                                                                        |
| Subscribe trigger         | <b>Lazy</b> — auto-subscribe just before the first payout if the account doesn't exist yet (validate → subscribe → transfer).                                                                                                                                                        |
| Threshold model           | One <code>MinimumPayoutAmount</code> per <code>WalletType</code> , admin-editable, USD.                                                                                                                                                                                              |
| Audit account capture     | Immutable <b>snapshot</b> of gateway + payout account, copied (not FK'd) onto <code>PayoutAttempt</code> at payment time.                                                                                                                                                            |
| Audit layering            | Two layers: <code>PayoutAttempt</code> (one high-level row per payment) + <code>MemberWalletApiLog</code> (raw per-HTTP-call log).                                                                                                                                                   |
| Receipt authenticity      | Internal Merkle hash-chain + periodic external blockchain anchoring of the Merkle root via <code>IDocumentAnchorService</code> (external anchor stubbed until creds).                                                                                                                |
| Email                     | Reuse <code>IService</code> ; auto-send toggle via <code>GlobalParameters</code> ; manual resend endpoint.                                                                                                                                                                           |
| Credentials               | Real contract + service with <code>ApiCredential</code> resolution; runs in simulated/switchable mode until i-payout/Volet creds exist (same pattern as Spreedly).                                                                                                                   |
| Bulk payout               | Admin can choose, per gateway, to <b>export a CSV</b> for a portal-side bulk transfer instead of online API calls (needed at promo-scale volume). Online (per-member) and CSV bulk coexist.                                                                                          |
| CSV reconciliation        | <b>Both</b> modes: upload the gateway's results file for per-row reconciliation (Paid only the rows the gateway confirms; failed rows released back to Pending with audited error), <b>with</b> a manual bulk mark-paid fallback when no results file exists. Admin picks per batch. |
| Bulk audit/receipt        | CSV-paid payments get the <b>same</b> immutable snapshot, receipt, hash-chain and email as online payments — snapshot frozen at <b>export</b> time.                                                                                                                                  |

### 4. Reuse map (existing code to build on)

- **Outbound vs inbound:** `IGatewayService` (Charge/Refund) and the Spreedly routing engine are *inbound* (charging). Payouts get a **new, separate** abstraction — do not overload `IGatewayService` (ISP).
- **Wallets:** `MemberProfilesWallet` ( `WalletType`, `AccountIdentifier`, `IsPreferred`, `eWalletPasswordEncrypted` ), `MemberProfilesWalletHistory`. `WalletType` enum already has `eWallet=4`, `Volet=6`.
- **Raw API audit:** `MemberWalletApiLog` already models  
Operation/Endpoint/HttpMethod/RequestBody/ResponseBody/HttpStatusCode/Success/ErrorMessage/DurationMs.
- **Earnings:** `CommissionEarning` ( `BeneficiaryMemberId`, `Status`, `PaymentDate`, `Amount` ). Pending balance summed in `CommissionBalanceService` / `CommissionsService.GetSummaryAsync`.
- **Existing payout flow:** `PayoutHandler`, `PayoutCommand`, `CommissionPayoutJob` (weekly Friday). These get refactored to go through the new orchestrator.
- **PDF:** `ITextCertificatePdfFillerService` (iText7 canvas overlay), `ICertificateStorage` / `LocalCertificateStorage` (S3 deferred), `CertificateFileNaming.Sha256Hex`.
- **Email:** `IService` (template + variables), `EmailTemplate` / `EmailTemplateLocalization`, `NotificationEvents`. Real transport not wired (`NullEmailService`) — fine.

- **Admin config:** `GlobalParameter` key/value + `AdminRecurringBillingController` GET/PUT pattern.
- **Credentials:** `ApiCredential` (ServiceKey + encrypted key) — used by Spreadly.
- **Gaps confirmed:** no per-gateway thresholds, no payout-gateway abstraction, no blockchain/Merkle/anchor code.

## 5. Iteration 1 — Payout Gateway + audit core

### 5.1 Domain (new)

`PayoutGatewaySetting` : `AuditChangesIntKey`

```
WalletType WalletType      (unique)
decimal    MinimumPayoutAmount // USD
bool       IsActive
string?    Notes
```

`PayoutAttempt` : `AuditChangesLongKey` — one row per payment attempt to one ambassador; immutable once written.

```
string      MemberId
WalletType  WalletTypeSnapshot
string      PayoutAccountSnapshot // crypto address / email – copied, not FK
string?     PayoutAccountMetaSnapshot // JSON: crypto network, currency, country, etc.
decimal     AmountUsd
DateTime    ProcessDateUtc // admin-defined "payment date"
string      Outcome // Pending | Success | Failed
string?     GatewayTransactionId
string?     GatewayErrorCode
string?     GatewayErrorMessage
DateTime    AttemptedAtUtc
DateTime?   CompletedAtUtc
long?       LatencyMs
int         EarningsCount
DisbursementMode DisbursementMode // Online | CsvBulk
string?     PayoutBatchId // null for Online; FK to PayoutBatch for CsvBulk
// Receipt fields (Iteration 3, nullable until then):
string?     ReceiptUrl
string?     ReceiptSha256
long?       ReceiptLedgerSeq
string?     ReceiptPrevHash
string?     ReceiptAnchorRef
```

`PayoutAttemptEarning` : `AuditChangesLongKey` — immutable join of which earnings entered a payment. Also **reserves** the earning while a CSV batch is in flight (prevents double-payment).

```
long      PayoutAttemptId
string    CommissionEarningId
decimal   Amount
```

`PayoutBatch : AuditChangesStringKey` — groups one CSV bulk export (Iteration 4; entity created in Sprint 16 to avoid a second migration).

```
WalletType WalletType
DateTime ProcessDateUtc
string Status // Exported | Reconciled | PartiallyReconciled | Cancelled
string? ExportCsvUrl
string? ResultCsvUrl
int MemberCount
decimal TotalAmountUsd
string? ReconciledBy
DateTime? ReconciledAt
string? Notes
```

`enum DisbursementMode { Online = 1, CsvBulk = 2 }` (Domain/Enums).

Indexes: `PayoutAttempt(MemberId, ProcessDateUtc)`, `PayoutAttempt(Outcome, ProcessDateUtc)`, `PayoutAttempt(PayoutBatchId)`, `PayoutAttemptEarning(PayoutAttemptId)`, `PayoutAttemptEarning(CommissionEarningId)`, `PayoutGatewaySetting(WalletType)` unique, `PayoutBatch(WalletType, ProcessDateUtc)`.

## 5.2 Gateway abstraction (Billing project)

```
public interface IPayoutGatewayService
{
    WalletType GatewayType { get; }
    Task<Result<PayoutAccountResult>> SubscribeAccountAsync(PayoutAccountContext ctx, CancellationToken ct);
    Task<Result<PayoutAccountResult>> ValidateAccountAsync(PayoutAccountContext ctx, CancellationToken ct);
    Task<Result<PayoutBalanceResult>> GetBalanceAsync(string memberId, CancellationToken ct = default);
    Task<Result<PayoutTransferResult>> DisburseAsync(PayoutTransferContext ctx, CancellationToken ct = default);
}
```

- Result types each carry `GatewayCode` + `GatewayMessage` (raw provider strings) so failures are fully audited.
- Implementations: `EWalletPayoutGatewayService` (i-payout, `WalletType.eWallet`), `VoletPayoutGatewayService` (`WalletType.Volet`). Both resolve credentials via `ApiCredential` and run in switchable simulated mode until creds exist.
- `IPayoutGatewayResolver.Resolve(WalletType)` — mirrors `CardGatewayResolver`. Unknown/disabled type → failure result, not exception.

## 5.3 Payout orchestrator

`IPayoutOrchestrator.ExecutePayoutAsync(memberId, processDateUtc, ct)`: 1. Load member's preferred/active wallet → resolve gateway by `WalletType`. 2. Gather pending `CommissionEarning` for member with `PaymentDate ≤ processDate`, `Status=Pending`, `!IsDeleted`. Sum = amount. 3. **Lazy subscribe:** `ValidateAccountAsync`; if not found → `SubscribeAccountAsync`. Each call writes a `MemberWalletApiLog`. 4. Create `PayoutAttempt` (Outcome=Pending) snapshotting `WalletType` + `AccountIdentifier` (+ meta). 5. `DisburseAsync`. Write `MemberWalletApiLog` for the transfer call. 6. **On success:** call the shared `FinalizePayoutSuccess(attempt)` routine — mark earnings `Paid` (`PaymentDate` /audit updated), write `PayoutAttemptEarning` rows, set attempt Outcome=Success + `GatewayTransactionId` + `CompletedAtUtc` + `LatencyMs`. (Receipt + email hook here in Iteration 3; CSV reconciliation reuses this same routine in Iteration 4.) 7. **On failure:** earnings stay `Pending`; attempt

Outcome=Failed + `GatewayErrorCode` / `GatewayErrorMessage` . No partial payment. 8. All DB writes in one transaction (UoW).

**Earnings reservation / double-pay guard:** candidate queries (online and CSV export) must exclude earnings already attached to a non-failed `PayoutAttempt` (an open online attempt or an un-reconciled CSV batch). This is what prevents the same earning from being paid twice across modes.

Refactor `PayoutHandler` + `CommissionPayoutJob` to call the orchestrator, so the weekly job and the dashboard share one gateway-confirmed path. **Behavior change:** a gateway rejection now leaves earnings `Pending` instead of silently marking `Paid` .

## 5.4 Endpoints (Iteration 1)

```
GET  /api/v1/admin/members/{memberId}/payout/balance    // ambassador gateway balance (live)
POST /api/v1/admin/members/{memberId}/payout/validate  // validate account exists at gateway
GET  /api/v1/admin/payout-gateway-settings             // list thresholds
PUT  /api/v1/admin/payout-gateway-settings/{walletType} // set MinimumPayoutAmount / IsActive
POST /api/v1/admin/members/{memberId}/payout          // process payout (orchestrator) – used by dashb
```

Auth: `SuperAdmin`, `Admin`, `BillingManager` . Rate-limited per the §3.5 sensitive-endpoint policy.

## 6. Iteration 2 — Payments dashboard

### 6.1 Endpoints

```
GET  /api/v1/admin/payouts/pending    // ?processDate=&walletType=&page=&pageSize=
GET  /api/v1/admin/payouts/stats      // ?processDate=
POST /api/v1/admin/payouts/{memberId}/pay // body: { processDate } → orchestrator (Iteration 1)
```

**/pending candidate rule:** members whose `SUM(pending CommissionEarning where PaymentDate ≤ processDate) ≥ MinimumPayoutAmount` of their preferred wallet's `WalletType` (setting must be `IsActive` ). Each row:

```
MemberId, FullName, PendingAmount, WalletType,
LastAttemptOutcome, LastAttemptError    // from latest PayoutAttempt for (member, processDate)
ProfileCommissionsPendingUrl           // deep link → admin profile > Commissions tab > Pending section
```

Optional `walletType` filter for processing one gateway at a time.

**/stats :** per `WalletType` → `PendingTotal` (candidates' due amounts) and `PaidTodayTotal` (sum of `PayoutAttempt` Success with `CompletedAtUtc` = today). Lets the admin see at a glance whether a gateway's batch already ran today.

### 6.2 Frontend (Jazmin-FrontEnd, AdminWeb Blazor)

Top stat cards per gateway (pending vs paid-today). Filter by gateway. Table rows with full name (deep link), pending amount, gateway, pay button. After processing: green check on success, red cross on failure + comment icon → popup showing `GatewayErrorCode` / `GatewayErrorMessage` from the latest `PayoutAttempt` .

## 7. Iteration 3 — Receipts, authenticity & audit screen

### 7.1 Receipt PDF + storage

- `IReceiptPdfRenderer.RenderAsync(PayoutReceiptData)` — iText7, mirrors `ITextCertificatePdfFillerService`. Content: ambassador name/id, gateway + **account snapshot**, amount, process date, gateway transaction id, itemized earnings (from `PayoutAttemptEarning`), receipt id, SHA-256, anchor reference.
- `IReceiptStorage` / `LocalReceiptStorage` mirroring `ICertificateStorage` (S3 deferred). Filename via shared `Sha256Hex`.
- Generated on successful payout (Iteration 1 hook point). URL + hash stored on `PayoutAttempt`.

### 7.2 Authenticity — internal Merkle hash-chain + external anchor

- On receipt creation: `ReceiptSha256 = SHA256(pdf bytes)`; `ReceiptLedgerSeq = prev + 1`; `ReceiptPrevHash = SHA256(prevHash || ReceiptSha256)` (chained — any tampering breaks the chain).
- `IDocumentAnchorService.AnchorAsync(merkleRoot)` — abstraction; **stub** (records intent, returns simulated ref) until a public-chain provider (OpenTimestamps/Bitcoin or Polygon) and creds exist.
- HangFire job `ReceiptAnchorJob` (queue `billing`, e.g. daily) computes the Merkle root over un-anchored receipts, calls the anchor service, writes `ReceiptAnchorRef` back. Idempotent/resumable.
- Verification endpoint recomputes the PDF hash, re-walks the chain, and reports anchor status.

### 7.3 Email (auto-send + resend)

- `NotificationEvents.PayoutReceiptIssued` + seeded `EmailTemplate` (localized). Variables: name, amount, gateway, date, receipt link.
- Auto-send toggle: `GlobalParameter` key `PayoutReceiptEmailAutoSend` (default `true`), read in the orchestrator hook; admin GET/PUT to flip it.
- `POST /api/v1/admin/payouts/{attemptId}/receipt/resend` — manual resend when an ambassador claims non-receipt. Each send tracked.

### 7.4 Audit screen endpoints

```
GET /api/v1/admin/payouts/audit // ?from=&to=&memberId=&walletType=&outcome=&page=
GET /api/v1/admin/payouts/audit/{attemptId} // detail: snapshot, earnings, receipt, anchor
GET /api/v1/admin/payouts/audit/{attemptId}/gateway-log // MemberWalletApiLog history (raw responses + int
GET /api/v1/admin/payouts/audit/{attemptId}/receipt // download
GET /api/v1/admin/payouts/audit/{attemptId}/verify // recompute hash + chain + anchor status
POST ... /receipt/resend // (7.3)
```

Frontend (Jazmin): filterable audit table (date range / ambassador / gateway / outcome), detail drawer with the immutable snapshot, itemized earnings, full gateway-response history, receipt download, and an authenticity badge.

## 8. Iteration 4 — Bulk CSV payout path

For promo-scale windows where per-member API disbursement is too slow, the admin can process a whole gateway's pending payouts as one CSV the gateway portal ingests, then reconcile the results back.

### 8.1 Per-gateway CSV adapters

- `IPayoutCsvFormatter` (resolved by `WalletType`) — builds the export file in each provider's column layout (i-payout and Volet differ). Every row carries the `PayoutAttempt.Id` as an export reference so the results file can be matched back deterministically.
- `IPayoutResultCsvParser` (resolved by `WalletType`) — parses the gateway's results file (per-row success/failure + code) back to attempts.

## 8.2 Export flow ( `POST ... /payouts/batches/export` )

1. Resolve candidates for (`walletType`, `processDate`) using the **same threshold + reservation rule** as the dashboard, excluding already-reserved earnings.
2. Create a `PayoutBatch` (Status=Exported) and one `PayoutAttempt` per member with `DisbursementMode=CsvBulk`, `Outcome=Pending`, `PayoutBatchId` set, **account/gateway snapshot frozen now**, and `PayoutAttemptEarning` rows reserving the earnings.
3. Render the CSV via the gateway's formatter, store via `IReceiptStorage` /file storage, set `ExportCsvUrl`, stream the file to the admin.
4. Earnings are **not** Paid yet — only reserved.

## 8.3 Reconciliation

- **Per-row ( `POST ... /batches/{id}/reconcile` , multipart results file):** parse with the gateway's parser; for each matched attempt → success rows call `FinalizePayoutSuccess` (mark Paid + receipt + hash-chain + email); failure rows set Outcome=Failed, release the reserved earnings back to `Pending`, audit `GatewayErrorCode` / `GatewayErrorMessage`. Batch → Reconciled or PartiallyReconciled. Store `ResultCsvUrl`.
- **Bulk fallback ( `POST ... /batches/{id}/mark-paid` ):** when the gateway returns no results file, after the admin confirms the portal processed the batch, mark every attempt in the batch Success via `FinalizePayoutSuccess`. Batch → Reconciled.
- **Cancel ( `POST ... /batches/{id}/cancel` ):** release all reserved earnings back to `Pending`, set attempts Failed/Cancelled, Batch → Cancelled. For batches uploaded in error before any portal processing.

## 8.4 Endpoints (Iteration 4)

```
POST /api/v1/admin/payouts/batches/export           // { walletType, processDate } → creates batch + res
GET  /api/v1/admin/payouts/batches                 // ?status=&walletType=&from=&to=&page=
GET  /api/v1/admin/payouts/batches/{id}            // detail + per-member attempts
GET  /api/v1/admin/payouts/batches/{id}/csv        // re-download export CSV
POST /api/v1/admin/payouts/batches/{id}/reconcile  // multipart results file → per-row reconcile
POST /api/v1/admin/payouts/batches/{id}/mark-paid  // bulk fallback
POST /api/v1/admin/payouts/batches/{id}/cancel     // release reservations
```

Auth: `SuperAdmin`, `Admin`, `BillingManager`.

## 8.5 Frontend (Jazmin)

Dashboard gains an **"Export CSV"** action per gateway (alongside per-row online pay). A **Batches** view lists batches with status, totals, download, and a reconcile uploader (drag results file) plus a guarded bulk mark-paid button. Batch detail shows per-member outcomes and links into the audit screen.

## 9. Cross-cutting rules



- All service methods return `Result<T>`; no exceptions to controllers. `CancellationToken ct = default` last param.
- Use `IDateTimeProvider` — never `DateTime.UtcNow` directly. (Note: `AuditInterceptor` overwrites `CreationDate` on insert with the real clock; date-window tests must re-save as Modified — see existing reference note.)
- No business logic in controllers. CQRS via MediatR.
- Migrations generated **and applied** (local conn string) for: `PayoutGatewaySetting`, `PayoutAttempt`, `PayoutAttemptEarning`, any `MemberWalletApiLog` additions.
- HangFire `ReceiptAnchorJob` carries `[Queue("billing")]` (per the per-service-queue rule).
- Snapshots are immutable: never update `WalletTypeSnapshot` / `PayoutAccountSnapshot` / `PayoutAttemptEarning` after creation.

---

## 10. Testing (Eyaelkys) — required cases per iteration

---

**Iteration 1** - `Payout_ReservedEarnings_ExcludedFromNewCandidates` (double-pay guard) -

`Payout_WhenGatewayConfirms_MarksEarningsPaid` -  
`Payout_WhenGatewayRejects_LeavesEarningsPending_AndAuditsError` -  
`Payout_WhenAccountMissing_AutoSubscribesThenTransfers` -  
`Payout_SnapshotsGatewayAndAccount_AtPaymentTime` -  
`Payout_WhenAccountChangesLater_AuditSnapshotUnchanged` -  
`PayoutAttempt_GroupsEarnings_WritesPayoutAttemptEarningRows` - `Threshold_BelowMinimum_NotEligible` / `Threshold_AtOrAboveMinimum_Eligible` - `Resolver_UnknownWalletType_ReturnsFailureNotException` -  
`GetBalance_ReturnsAmbassadorGatewayBalance`

**Iteration 2** - `PendingDashboard_OnlyReturnsCandidatesAtOrAboveThreshold` -

`PendingDashboard_FiltersByWalletType` -  
`PendingDashboard_AmountCountsOnlyEarningsDueOnOrBeforeProcessDate` -  
`Stats_PaidTodayReflectsSuccessfulAttemptsToday` - `PayIndividual_OnSuccess_RowShowsSuccess` / `PayIndividual_OnFailure_RowShowsErrorReason`

**Iteration 3** - `Receipt_GeneratedOnSuccessfulPayout_StoresUrlAndHash` -

`HashChain_TamperedReceipt_FailsVerification` - `HashChain_SequenceAndPrevHash_AreChained` -  
`Anchor_Stub_RecordsRefAndMarksAnchored` - `EmailAutoSend_WhenToggleOn_SendsReceipt` / `WhenToggleOff_DoesNotSend` - `ReceiptResend_SendsAgain_AndTracks` -  
`Audit_FiltersByDateRange_Ambassador_Gateway` - `Verify_RecomputesHash_AndReportsAnchorStatus`

**Iteration 4** - `Export_CreatesBatch_ReservesEarnings_DoesNotMarkPaid` - `Export_UsesGatewaySpecificColumns` (i-

payout vs Volet) - `Export_ExcludesEarningsReservedByAnotherBatch` -  
`Reconcile_SuccessRows_FinalizePaid_FailRows_ReleasedToPending` -  
`Reconcile_PartialResults_SetsBatchPartiallyReconciled` - `MarkPaidFallback_MarksAllAttemptsPaid` -  
`CancelBatch_ReleasesReservations_ToPending` - `CsvPaid_GeneratesReceipt_WithExportTimeSnapshot`

Each business rule gets a positive + negative case. 100% pass before any commit suggestion. ≥80% coverage on touched projects.

---

## 11. Open items (confirm before/at implementation)

---

- i-payout / Volet sandbox credentials availability (determines real HTTP vs simulated mode at build time).
- External anchor provider choice (OpenTimestamps/Bitcoin vs Polygon/Ethereum) — only affects the `IDocumentAnchorService` stub's eventual real impl.



- Receipt PDF visual template (Jazmin) — layout/branding.
- Exact i-payout / Volet bulk-upload CSV column specs and results-file formats (drives `IPayoutCsvFormatter` / `IPayoutResultCsvParser` per gateway).